MANIPAL ACADEMY OF HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL INSTITUTE OF TECHNOLOGY

School of Computer Engineering

CES 2112 Digital Systems Lab

(Open Ended Experiment Report)

on

DESIGN AND IMPLEMENTATION OF A BCD ARITHMETIC UNIT WITH EXCESS-3 CODE CONVERSION

Submitted by

Team member name	Section	Registration / Roll No.
Riya Agrawal	CSE – I, Batch 2	251090052030 / 46
Apurva Nayak	CSE – I, Batch 2	251090051928 / 44
Shreya Nirgide	CSE – I, Batch 2	251090052058 / 47
Aryan Achary	CSE – I, Batch 2	251090052730 / 61

Under the guidance of:

Dr Anoop B N, Assistant Professor (Senior Scale)

School of Computer Engineering, MIT Manipal

Date: 19/10/2026

TABLE OF CONTENTS

1. Problem Statement.....3
2. Objectives.....4
3. System Design and Block Diagram.....5
4. Verilog HDL Implementation.....8
5. Simulation, Verification and Results.....11
6. Individual Contribution.....13
7. Conclusion and Future Scope.....13
References.....14

1. PROBLEM STATEMENT

1.1 Problem statement

Context and motivation. Humans count, price and measure in decimal, while digital hardware computes in binary. Many everyday systems — calculators, billing and point-of-sale terminals, digital meters, clocks and seven-segment displays — therefore keep their data in Binary Coded Decimal (BCD), where each decimal digit is stored as its own 4-bit group. Decimal hardware is also important in commercial and financial computing, because binary fractions cannot represent values such as 0.1 exactly and the resulting rounding errors are unacceptable in monetary calculations [1]; decimal formats are now part of the IEEE 754 floating-point standard [2].

Existing limitation. A plain 4-bit binary adder does not produce valid BCD. When the sum of two BCD digits exceeds 9 (for example $7 + 5 = 1100_2$), the result falls into one of the six unused codes 1010–1111 or overflows, and a decimal correction of +0110 must be applied [3]. Subtraction adds a further difficulty: two's-complement subtraction of digits gives a wrapped binary pattern whenever the result is negative, which must be converted back to a sign and a decimal magnitude. Finally, many decimal systems also use the Excess-3 code, a self-complementing code that simplifies 9's-complement arithmetic; obtaining it needs a separate code converter.

Proposed solution. This project designs a single-digit BCD arithmetic unit that adds or subtracts two BCD digits under the control of a mode input M, applies the decimal correction automatically, reports negative differences in sign-magnitude form, and converts every result digit to Excess-3. The design is written as a hierarchy of small Verilog modules (full adder → 4-bit ripple-carry adder → BCD adder/subtractor → code converter → top module) and verified by exhaustive simulation.

Scope and assumptions. The inputs are two 4-bit BCD digits A and B (0–9) and a 1-bit mode M (0 = add, 1 = subtract). The outputs are a tens digit and a units digit in BCD and in Excess-3, a sign bit and an error flag. The circuit is purely combinational, so there is no clock or reset. Inputs outside the BCD range (10–15) are flagged through err, and all data outputs are then forced to 0. Multi-digit cascading, physical FPGA implementation and timing closure are outside the scope of this work.

1.2 System requirements

The design must satisfy the requirements in Table I. Each requirement is linked to the method used to verify it in Section 5.

TABLE I. SYSTEM REQUIREMENTS

ID	Requirement	Verification method
R1	With M = 0, produce the BCD sum A + B (0–18) as a tens digit and a valid units digit.	Exhaustive self-checking simulation (100 cases)
R2	With M = 1, produce |A − B| as a valid BCD digit, with sign = 1 when A < B.	Exhaustive self-checking simulation (100 cases)
R3	Convert each result digit to Excess-3 (BCD + 0011).	Checked in every simulated case; truth table
R4	Assert err and force every data output to 0 when A or B is not a valid BCD digit (> 9).	Exhaustive sweep of all 312 invalid combinations; T9, T10
R5	Purely combinational: outputs depend only on the present inputs (no clock, reset or stored state).	Code inspection (continuous assignments only); waveform (Fig. 2)

1.3 Relevance to the Sustainable Development Goals

The United Nations 2030 Agenda defines 17 Sustainable Development Goals (SDGs) [4]. Although this is a small laboratory design, it relates directly to the three goals listed in Table II.

TABLE II. MAPPING OF THE PROJECT TO UN SUSTAINABLE DEVELOPMENT GOALS

SDG	**How the project contributes**
SDG 4 – Quality Education	The modules are small, readable and fully tested, so they can be reused as teaching material for number codes, decimal correction and hierarchical HDL design in undergraduate digital-systems labs.
SDG 9 – Industry, Innovation and Infrastructure	Exact decimal arithmetic is a building block of billing terminals, energy and water meters, and financial infrastructure, where binary rounding errors are unacceptable [1]. A verified, reusable BCD unit supports such reliable digital infrastructure.
SDG 12 – Responsible Consumption and Production	The design was verified entirely in simulation before any hardware is built, which avoids repeated breadboard or FPGA prototyping and the electronic waste and component use that come with it. The compact design (two shared 4-bit adders) also keeps the gate count low.

2. OBJECTIVES

The objectives below follow directly from the problem statement and requirements R1–R5.

2.1 Primary objective

To design, implement in Verilog HDL and verify a combinational single-digit BCD adder/subtractor whose results are also presented in Excess-3 code.

2.2 Specific objectives

- Implement a 4-bit ripple-carry adder from full adders and reuse it for both addition and two's-complement subtraction, selected by the mode input M.
- Derive and implement the decimal-correction condition for addition and the sign-magnitude recovery logic for negative differences.
- Derive a minimised gate-level BCD-to-Excess-3 converter from its truth table using Karnaugh maps.
- Verify every possible input combination ($16 \times 16 \times 2 = 512$, of which 200 are valid and 312 invalid) with a self-checking testbench, and target zero mismatches.
- Keep the design compact by sharing two 4-bit adders across both operations instead of building separate adder and subtractor datapaths.

2.3 Scope and deliverables

- **Verilog design modules:** full_adder, rca4, bcd_addsub, bcd_to_xs3 and the top module bcd_alu_xs3.
- **Verification:** self-checking testbench tb_bcd_alu_xs3, simulation log and VCD waveform.
- **Documentation:** block diagram, truth table, test-case table, waveform figure and this report.

3. SYSTEM DESIGN AND BLOCK DIAGRAM

The system is a single combinational datapath with three functional parts: a shared binary adder/subtractor (Stage 1), a correction stage that turns the binary result into a valid BCD digit (Stage 2), and two code converters that produce the Excess-3 outputs. A separate validity checker watches the inputs and, through an output mask, forces all data outputs to 0 when an input is not a valid BCD digit.

3.1 Proposed architecture

Fig. 1 shows the top-level architecture. Operand B first passes through an XOR array controlled by M: when M = 0 it is passed unchanged, and when M = 1 it is inverted. Stage 1 is a 4-bit ripple-carry adder whose carry-in is also M, so it computes A + B for addition and A + B′ + 1 = A − B for subtraction. Its 4-bit result S1 and carry C1 go to the correction logic. That logic decides whether Stage 2 should add 0110 (addition with a result above 9) or form the two's complement of S1 (negative difference). Stage 2 is a second 4-bit ripple-carry adder whose output is the units digit. The tens digit and the sign come directly from the correction logic. Both result digits feed identical BCD-to-Excess-3 converters. Finally, every data output passes through the output mask, which ANDs it with err′.

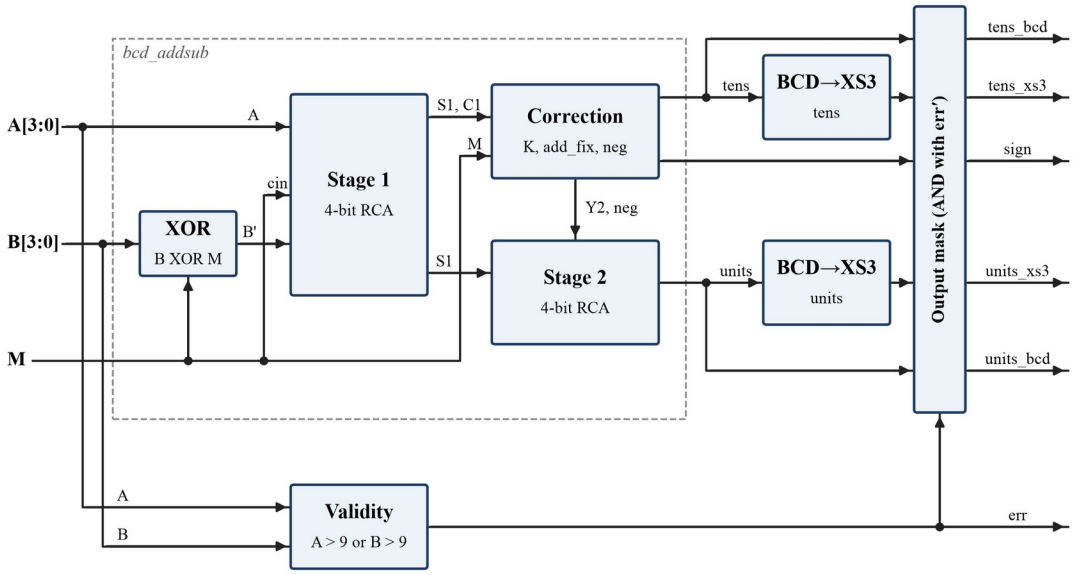


Fig. 1. Overall block diagram of the proposed BCD arithmetic unit (bcd_alu_xs3), including the output mask.

For addition, a correction is needed when the binary sum exceeds 9. This happens when Stage 1 produces a carry, or when the 4-bit sum is 1010–1111. The condition is [3]:

$$K = C1 + S1_3 \cdot S1_2 + S1_3 \cdot S1_1 \quad (1)$$

where C1 is the Stage-1 carry and $S1_3 \ldots S1_0$ are the sum bits. When K = 1 and M = 0, Stage 2 adds 0110 and the tens digit becomes 1. For subtraction, Stage 1 evaluates

$$A - B = A + B' + 1 \quad (2)$$

where the prime (′) denotes the bitwise complement. If A ≥ B the adder produces a carry (C1 = 1) and S1 is already the correct difference. If A < B there is no carry, S1 holds the two's complement of the

magnitude, and the correction logic sets neg = M·C1′. Stage 2 then computes S1′ + 1 to recover |A − B|, and sign is set to 1. Excess-3 is defined as

$$XS3 = BCD + 0011 \tag{3}$$

Table III gives the converter truth table. Minimising each output over the don't-care inputs 1010–1111 with Karnaugh maps [5] gives:

$$E_3 = B_3 + B_2(B_1 + B_0) \tag{4}$$

$$E_2 = B_2 \oplus (B_1 + B_0) \tag{5}$$

$$E_1 = (B_1 \oplus B_0)', \quad E_0 = B_0' \tag{6}$$

TABLE III. BCD TO EXCESS-3 CONVERSION TRUTH TABLE

Decimal	BCD (B3 B2 B1 B0)	Excess-3 (E3 E2 E1 E0)
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

3.2 Module description

The design is split into the five Verilog modules described in Table IV.

TABLE IV. MODULE DESCRIPTION

Module	Inputs	Outputs	Function
bcd_alu_xs3 (top)	a[3:0], b[3:0], m	tens_bcd, units_bcd, sign, tens_xs3, units_xs3, err	Connects the arithmetic core, both code converters, the validity check and the output mask.
bcd_addsub	a[3:0], b[3:0], m	units[3:0], tens, sign	Two-stage BCD adder/subtractor with decimal correction and sign-magnitude output.
rca4	a[3:0], b[3:0], cin	s[3:0], cout	4-bit ripple-carry adder built from four full adders; used twice.
full_adder	a, b, cin	s, cout	1-bit full adder: s = a⊕b⊕cin, cout = ab + cin(a⊕b).
bcd_to_xs3	bcd[3:0]	xs3[3:0]	Gate-level BCD-to-Excess-3 converter from Eqs. (4)–(6).

3.3 Design decisions

- **Combinational design.** Addition, subtraction and code conversion depend only on the present inputs, so no clock, reset or state machine is used. The outputs settle after the gate propagation delay.
- **Shared adders.** Instead of separate adder and subtractor circuits, a single XOR array and carry-in M turn Stage 1 into an adder/subtractor. Stage 2 is also shared: its operands are selected so that one adder performs both the +0110 correction and the two's-complement recovery. The whole arithmetic core uses only eight full adders.
- **Sign-magnitude output.** A negative difference is reported as a sign bit plus a BCD magnitude, so the units digit is always a valid BCD digit that can drive a display directly.

- **Ripple-carry adders.** For 4-bit operands, a ripple-carry adder is the smallest and simplest option. A carry-lookahead adder would save very little delay at this width.
- **Structural hierarchy.** Each block is a separate module, so it can be tested on its own and reused when the design is extended to several digits.

4. VERILOG HDL IMPLEMENTATION

All modules are written in synthesizable Verilog-2005 [6] using continuous assignments and structural instantiation [7]. The complete design is listed below.

4.1 Design module

Listing 1: full_adder and rca4.

```
// 1-bit full adder (dataflow)
module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire s,
    output wire cout
);
    assign s    = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

// 4-bit ripple-carry adder built from four full adders
module rca4 (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       cin,
    output wire [3:0] s,
    output wire       cout
);
    wire c1, c2, c3;
    full_adder fa0 (.a(a[0]), .b(b[0]), .cin(cin), .s(s[0]), .cout(c1));
    full_adder fa1 (.a(a[1]), .b(b[1]), .cin(c1),  .s(s[1]), .cout(c2));
    full_adder fa2 (.a(a[2]), .b(b[2]), .cin(c2),  .s(s[2]), .cout(c3));
    full_adder fa3 (.a(a[3]), .b(b[3]), .cin(c3),  .s(s[3]), .cout(cout));
endmodule
```

Listing 2: bcd_addsub (arithmetic core).

```
// Single-digit BCD adder/subtractor
//   m = 0 : A + B  -> {tens, units} (0..18), sign = 0
//   m = 1 : A - B  -> sign-magnitude result, magnitude in units (0..9)
module bcd_addsub (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       m,       // 0 = add, 1 = subtract
    output wire [3:0] units,
    output wire       tens,    // decimal carry (tens digit, 0 or 1)
    output wire       sign     // 1 = negative difference
);
    // Stage 1: binary adder/subtractor (two's complement when m = 1)
    wire [3:0] b_x = b ^ {4{m}};
    wire [3:0] s1;
    wire       c1;
    rca4 stage1 (.a(a), .b(b_x), .cin(m), .s(s1), .cout(c1));

    // Decimal correction condition for addition: sum > 9
    wire k   = c1 | (s1[3] & s1[2]) | (s1[3] & s1[1]);
    wire add_fix = ~m & k;
    // Negative difference: no end-around carry out of stage 1
    wire neg = m & ~c1;

    // Stage 2: +0110 correction (add) or two's-complement recovery (negative sub)
    wire [3:0] x2 = s1 ^ {4{neg}};
    wire [3:0] y2 = {1'b0, add_fix, add_fix, 1'b0};
    wire       c2_unused;
```

```
    rca4 stage2 (.a(x2), .b(y2), .cin(neg), .s(units), .cout(c2_unused));

    assign tens = add_fix;
    assign sign = neg;
endmodule
```

Listing 3: bcd_to_xs3 and top module bcd_alu_xs3.

```
// BCD to Excess-3 code converter (gate-level, from K-map minimisation)
// Inputs 1010-1111 are don't-cares.
module bcd_to_xs3 (
    input  wire [3:0] bcd,
    output wire [3:0] xs3
);
    assign xs3[3] = bcd[3] | (bcd[2] & (bcd[1] | bcd[0]));
    assign xs3[2] = bcd[2] ^ (bcd[1] | bcd[0]);
    assign xs3[1] = ~(bcd[1] ^ bcd[0]);
    assign xs3[0] = ~bcd[0];
endmodule

// Top module: BCD arithmetic unit with Excess-3 output
module bcd_alu_xs3 (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire       m,          // 0 = add, 1 = subtract
    output wire [3:0] tens_bcd,
    output wire [3:0] units_bcd,
    output wire       sign,
    output wire [3:0] tens_xs3,
    output wire [3:0] units_xs3,
    output wire       err         // high when either input is not a valid BCD digit
);
    wire       tens, sign_r;
    wire [3:0] units_r, tens_r, tens_x, units_x;

    bcd_addsub u_arith (.a(a), .b(b), .m(m), .units(units_r), .tens(tens), .sign(sign_r));

    assign tens_r = {3'b000, tens};

    bcd_to_xs3 u_xs3_t (.bcd(tens_r),  .xs3(tens_x));
    bcd_to_xs3 u_xs3_u (.bcd(units_r), .xs3(units_x));

    // Input validity: 1010-1111 are not BCD digits
    assign err = (a[3] & (a[2] | a[1])) | (b[3] & (b[2] | b[1]));

    // Output mask: every data output is forced to 0 for an invalid input
    wire ok = ~err;
    assign tens_bcd  = tens_r  & {4{ok}};
    assign units_bcd = units_r & {4{ok}};
    assign sign      = sign_r  & ok;
    assign tens_xs3  = tens_x  & {4{ok}};
    assign units_xs3 = units_x & {4{ok}};
endmodule
```

4.2 Code explanation

- **full_adder / rca4:** The full adder implements the standard sum and carry equations. rca4 chains four instances through internal carries c1–c3.
- **Stage 1 (bcd_addsub):** b_x = b ^ {4{m}} replicates M across four bits to invert B only in subtract mode, and cin = m supplies the +1 of the two's complement. One adder therefore serves both operations.
- **Correction signals:** k implements Eq. (1). add_fix = ~m & k restricts the +0110 correction to addition, and neg = m & ~c1 detects a negative difference.

- **Stage 2:** x2 = s1 ^ {4{neg}} inverts S1 only for a negative difference, and cin = neg adds 1, giving S1′ + 1. y2 = {0, add_fix, add_fix, 0} equals 0110 only when the decimal correction is needed. In every other case Stage 2 passes S1 through unchanged. Stage 2's carry-out is not used: for valid inputs, the decimal carry is already captured by add_fix.
- **Outputs:** tens = add_fix and sign = neg. The top module widens tens to a 4-bit BCD digit (000T) so that both digits can use the same bcd_to_xs3 module.
- **Validity check:** a[3] & (a[2] | a[1]) is true exactly for 1010–1111, i.e., values above 9. The same test on B is ORed into err.
- **Output mask:** ok = ~err is replicated across each bus and ANDed with it, so an invalid input gives 0 on every data output instead of a meaningless value. The masked Excess-3 output 0000 is not a valid Excess-3 code, so the result cannot be mistaken for a real digit.

4.3 Testbench

The testbench tb_bcd_alu_xs3 is self-checking. A task named check computes the expected result behaviourally, using integer arithmetic and the operators / and %. The Excess-3 outputs are expected to be the digit + 3, and for an invalid input every data output is expected to be 0. All six DUT outputs are compared in one step with the case-inequality operator !==, so an unknown (X) or floating (Z) bit also counts as a failure. No clock is needed; each stimulus is held for 10 ns. The testbench first applies ten directed cases (T1–T10 in Table VI), then sweeps all 512 combinations of A, B (0–15) and M. Its essential part is:

```
task check;
  begin
    e_err = (a > 9) || (b > 9);
    if (e_err) begin          // invalid: all data outputs must be 0
      e_t = 0; e_u = 0; e_s = 0; e_tx = 0; e_ux = 0;
    end else begin
      if (m == 0) begin r = a + b; e_s = 0; end
      else begin r = (a >= b) ? a - b : b - a; e_s = (a < b); end
      e_t = r / 10;  e_u = r % 10;  e_tx = e_t + 3;  e_ux = e_u + 3;
    end
    // !== also fails on any X or Z output bit
    if ({err, sign, tens_bcd, units_bcd, tens_xs3, units_xs3} !==
        {e_err, e_s, e_t, e_u, e_tx, e_ux}) begin
      fail = fail + 1;
      $display("FAIL a=%0d b=%0d m=%0d", a, b, m);
    end else
      pass = pass + 1;
  end
endtask
...
for (k = 0; k < 2; k = k + 1)          // exhaustive sweep: 512 cases
  for (i = 0; i < 16; i = i + 1)
    for (j = 0; j < 16; j = j + 1) begin
      a = i; b = j; m = k; #10; check;
    end
```

5. SIMULATION, VERIFICATION AND RESULTS

The design was compiled with all warnings enabled and simulated with Icarus Verilog [8]. The testbench wrote a text log and a VCD waveform file. All values in this section come from that simulation run.

5.1 Experimental setup

Table V summarises the tools and the test set used for verification.

TABLE V. EXPERIMENTAL SETUP

Item	Details
HDL / language	Verilog-2005 (IEEE Std 1364-2005), compiled with iverilog -g2005 -Wall
Simulator / version	Icarus Verilog 13.0 (stable); waveform from the VCD dump
Synthesis tool / device (if used)	N/A — functional (zero-delay) simulation only
Test cases	10 directed cases (8 valid, 2 invalid) + exhaustive sweep of all 512 combinations (200 valid, 312 invalid)

5.2 Test cases and expected outcomes

Table VI lists the directed cases, chosen to cover each path through the design, and the exhaustive sweep. “Observed” values are copied from the simulation log as tens/units (BCD) and Excess-3.

TABLE VI. TEST CASES AND OBSERVED RESULTS

Test ID	Input / condition	Expected output	Observed result
T1	M=0, 3 + 4 (no correction)	07, sign 0; XS3 0011 1010	0000 0111; XS3 0011 1010 ✓
T2	M=0, 7 + 5 (sum 1100 > 9, +6)	12; XS3 0100 0101	0001 0010; XS3 0100 0101 ✓
T3	M=0, 8 + 8 (binary carry)	16; XS3 0100 1001	0001 0110; XS3 0100 1001 ✓
T4	M=0, 9 + 9 (maximum sum)	18; XS3 0100 1011	0001 1000; XS3 0100 1011 ✓
T5	M=1, 8 − 3 (positive)	+5; XS3 0011 1000	sign 0, 0101; XS3 0011 1000 ✓
T6	M=1, 2 − 7 (negative)	−5; XS3 0011 1000	sign 1, 0101; XS3 0011 1000 ✓
T7	M=1, 5 − 5 (zero)	+0; XS3 0011 0011	sign 0, 0000; XS3 0011 0011 ✓
T8	M=0, 0 + 0 (all zero)	00; XS3 0011 0011	0000 0000; XS3 0011 0011 ✓
T9	M=0, A = 12 (invalid)	err = 1; all data outputs 0	err = 1; 0000 0000; XS3 0000 0000 ✓
T10	M=1, B = 10 (invalid)	err = 1; all data outputs 0	err = 1; sign 0; 0000 0000; XS3 0000 0000 ✓
T11	Sweep: A, B ∈ 0–9, M ∈ {0, 1} (valid)	200 correct results	200 passed, 0 failed ✓
T12	Sweep: A or B ∈ 10–15, M ∈ {0, 1} (invalid)	312 × (err = 1, outputs 0)	312 passed, 0 failed ✓

5.3 Simulation waveform

Fig. 2 shows the waveform for the directed cases T1–T10 (0–100 ns). Multi-bit buses are shown in decimal for the BCD signals and in binary for the Excess-3 signals.

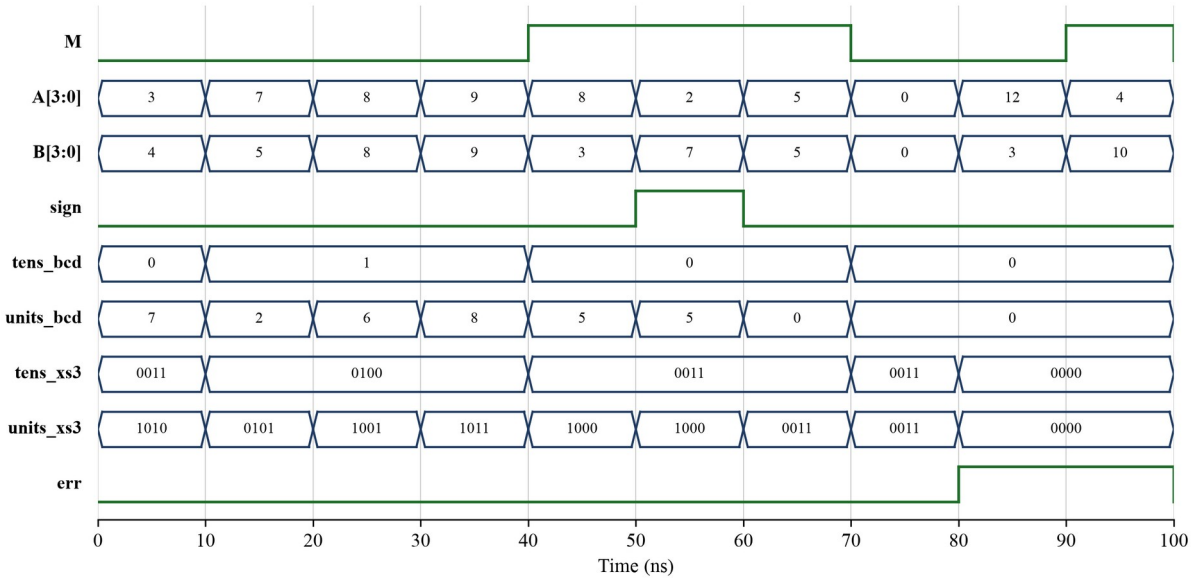


Fig. 2. Simulation waveform of the directed test cases T1–T10 (Icarus Verilog VCD output).

5.4 Results and discussion

All ten directed cases and all 512 exhaustive combinations matched the expected values. The simulator reported “Exhaustive result: 512 passed, 0 failed (200 valid, 312 invalid)”. This verifies R1–R3 for every valid input and R4 for every invalid one. The directed cases exercise each correction path. In T2 (7 + 5), Stage 1 produces the binary sum 1100, which K detects, and Stage 2 corrects it to 0010 with tens = 1. T3 (8 + 8) triggers the correction through the binary carry C1 instead. In T6 (2 − 7), Stage 1 leaves the wrapped value 1011, which Stage 2 converts to the magnitude 0101 with sign = 1. In T9 and T10, err rises as soon as an out-of-range digit appears, and Fig. 2 shows every data output, including both Excess-3 buses, dropping to 0.

To confirm that the testbench can actually detect errors, three faults were injected one at a time and the sweep was re-run: removing the $S1_3 \cdot S1_1$ term from K (17 failures), inverting the neg condition (100 failures) and complementing the E_1 output of the converter (200 failures). Every fault was caught, so the zero-failure result of the correct design is meaningful.

Because the simulation is zero-delay and functional, it confirms logical correctness, not real propagation delay. R5 holds by construction: every module uses only continuous assignments, and the outputs in Fig. 2 change only when the inputs change. The longest logic path runs through the XOR array, the Stage-1 carry chain, the correction logic, the Stage-2 carry chain, the converter and the mask, so the two ripple-carry chains dominate the delay. Measured timing and area would require synthesis to a target device, which is listed as future work.

6. INDIVIDUAL CONTRIBUTION

The work was divided into four parts of comparable size, as summarised in Table VII. Each member also wrote the report sections that cover their own part.

TABLE VII. INDIVIDUAL CONTRIBUTION OF TEAM MEMBERS

Team member	Section / registration no.	Individual tasks performed	Evidence / deliverables
Riya Agrawal	CSE – I, Batch 2, Reg. 251090052030, Roll 46	Designed the full adder and 4-bit ripple-carry adder; derived the BCD addition correction condition K (Eq. 1) and the +0110 path through Stage 2.	full_adder.v, rca4.v, add path of bcd_addsub.v; Sections 3.1 and 3.3
Apurva Nayak	CSE – I, Batch 2, Reg. 251090051928, Roll 44	Designed the subtraction path: XOR array, two's-complement carry-in, negative-result detection (neg) and sign-magnitude recovery in Stage 2.	Subtract path of bcd_addsub.v; Sections 1.1, 1.2 and 2
Shreya Nirgide	CSE – I, Batch 2, Reg. 251090052058, Roll 47	Built the BCD-to-Excess-3 truth table, did the K-map minimisation (Eqs. 4–6), and wrote the code converter, the input-validity check and the output mask.	bcd_to_xs3.v, err and mask logic; Table III; Sections 3.2, 4.2 and 1.3 (SDGs)
Aryan Achary	CSE – I, Batch 2, Reg. 251090052730, Roll 61	Integrated the top module; wrote the self-checking testbench; ran directed, exhaustive and fault-injection simulations; produced the block diagram and waveform figures.	bcd_alu_xs3.v, tb_bcd_alu_xs3.v, simulation log, Figs. 1–2; Sections 4.3 and 5

Contribution statement

The architecture, including the decision to share both 4-bit adders between addition and subtraction, was agreed jointly in team discussions before coding began. Each member then built and unit-tested their own modules (individual work, as listed in Table VII). Integration and the exhaustive simulation were done together, so that any mismatch could be traced back to the module owner; the final run showed none. The conclusion was written jointly, and all four members reviewed and proofread the complete report.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project designed and verified a combinational BCD arithmetic unit that adds or subtracts two decimal digits under a mode input. It applies the +0110 decimal correction, gives negative differences in sign-magnitude form, and converts each result digit to Excess-3 code. The design is a hierarchy of five Verilog modules built around two shared 4-bit ripple-carry adders. Self-checking simulation in Icarus Verilog passed all 512 possible input combinations (200 valid and 312 invalid) and all directed edge cases, and fault injection showed that the testbench detects real errors. All the specific objectives in Section 2.2 were achieved, and requirements R1–R5 were verified. The project shows how combinational logic, arithmetic circuits, code conversion and hierarchical HDL design combine into a small, useful digital system. Through its educational, infrastructure and simulation-first aspects, it also supports SDGs 4, 9 and 12.

7.2 Limitations and future scope

- **Single digit only:** operands are limited to 0–9. A natural extension is an n-digit unit that cascades BCD digit slices through the decimal carry and uses 9's/10's-complement subtraction across digits.

- **No timing or area data:** the design should be synthesized for an FPGA (e.g., with Yosys or Vivado) to measure LUT count and propagation delay, and tested on a board with switches and seven-segment displays.
- **Ripple-carry delay:** for wider multi-digit versions, a carry-lookahead or carry-skip BCD adder would reduce the critical path.
- **Further code conversions:** Excess-3-to-BCD, BCD-to-seven-segment and Gray-code converters could be added to the same output stage.

REFERENCES

[1] M. F. Cowlishaw, “Decimal floating-point: Algorism for computers,” in Proc. 16th IEEE Symp. Computer Arithmetic (ARITH-16), Santiago de Compostela, Spain, 2003, pp. 104–111, doi: 10.1109/ARITH.2003.1207666.

[2] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, Jul. 2019, doi: 10.1109/IEEESTD.2019.8766229.

[3] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. New York, NY, USA: Pearson, 2018.

[4] United Nations, “Transforming our world: The 2030 Agenda for Sustainable Development,” UN General Assembly Resolution A/RES/70/1, Oct. 2015. Accessed: Oct. 19, 2026. [Online]. Available: https://sdgs.un.org/2030agenda

[5] C. H. Roth Jr. and L. L. Kinney, *Fundamentals of Logic Design*, 7th ed. Stamford, CT, USA: Cengage Learning, 2014.

[6] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, Apr. 2006, doi: 10.1109/IEEESTD.2006.99495.

[7] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[8] S. Williams, “Icarus Verilog.” Accessed: Oct. 19, 2026. [Online]. Available: https://steveicarus.github.io/iverilog/